Session 2: Working with data — Summary sheet

Reading files

assign data to variable

```
genes <- read_delim(
  "my_analysis/data/genes.csv")
```

file path

Use `read_delim()` for any file, or specifically `read_csv()`/`read_tsv()`

Pipe

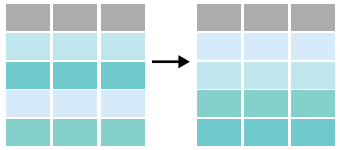
The pipe %>% allows you to chain together function calls, e.g. `round(mean(c(1,2,3)))` becomes:

```
c(1,2,3) %>% mean() %>% round()
```

%>% takes output & puts it as the **first argument** of the next function

Sorting

`arrange()`



```
data %>%
  arrange(col)
```

Sort `data` by the column `col`, in ascending order.

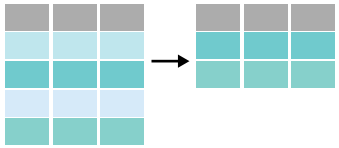
Use `desc()` to sort in descending order:

```
data %>% arrange(desc(col))
```

Use `slice_min(col, n = ...)` to find the *n* lowest values in `col`. Or use `slice_max()` for the top *n* values (beware of ties).

Filtering

`filter()`



```
data %>%
  filter(...)
```

Filter `data` to only include rows that pass this logical test (comparison), e.g.

```
data %>% filter(height > 1.5)
```

Use & (and), | (or) to combine tests:

```
data %>% filter(height > 1.5 &
  name == "bob" )
```

*height above 1.5 **and** name is 'bob'*

```
data %>% filter(height > 1.5 |
  name == "bob" )
```

*height above 1.5 **or** name is 'bob'*

Missing values

NA is used to represent missing values in R. NA are **contagious**: if you input NA you will get NA as output e.g. `mean(c(1,2,NA))` gives NA.

Remove rows containing NA values in a specific column `col` by filtering, e.g. `filter(!is.na(col))` or remove all NA in the data frame with `na.omit()`

Replace NA values with `replace_na()`

```
data %>% replace_na(
  list(col1 = "missing",
       col2 = 0))
```

in col2, replace with 0

in col1, replace with "missing"

Selecting columns

`select()`

```
data %>%
  select(col)
```

Select column `col` from `data` (output is a **data frame**). Use `pull()` if you want a **vector**.

or

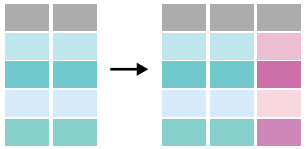
`pull()`

Functions to help you select:

not this column	`-col`
name includes	`contains("cm")`
name starts with	`starts_with("G")`

Modifying data

`mutate()`



```
data %>% mutate(
  new_col =
    col * 100)
```

Create column `new_col` in `data` that is equal to `col` multiplied by 100.

Use `case_when(condition ~ value)` to apply conditions, for example:

```
data %>% mutate(
  height_status = case_when(
    height > 2 ~ "tall",
    height < 1.5 ~ "short",
    .default = "regular"))
```

creates new column 'height_status' which has value 'tall' if height is more than 2, 'short' if height is less than 1.5 and 'regular' if neither of those conditions is met (i.e. height is between 1.5-2)